

École Supérieure des Communications de Tunis

Cahier des charges

ColdTrace : Supervision de la chaîne du froid

Projet Cloud of Things

Réalisé par :
Khalil Hedhili, Oussema Kard, Yessine Abdedayem

Encadré par :
Dr. Ing. Mohamed-Bécha Kaâniche

Année universitaire :
2026-2027

Table des matières

1 Concept **2**

1.1 Contexte . . . 2

1.2 Problématique . . . 2

1.3 Ambitions . . . 2

2 Clients, besoins et objectifs **2**

2.1 Clients et utilisateurs . . . 2

2.2 Besoins fonctionnels . . . 3

2.3 Besoins non fonctionnels . . . 3

2.4 Occasions d'utilisation . . . 3

2.5 Limites (hors périmètre) . . . 3

3 Technologies choisies **3**

3.1 Partie logicielle . . . 3

3.2 Partie matérielle . . . 4

4 Modèle commercial **5**

4.1 Stratégie de commercialisation . . . 5

4.2 Politique marketing (4 P) . . . 5

5 Livrables **5**

6 Contraintes **5**

6.1 Temps, argent, objectifs . . . 5

6.2 Méthodologie : eXtreme Programming . . . 6

6.3 Contraintes technologiques . . . 6

6.4 Estimation des charges (CoCoMo+) . . . 6

6.5 Planning (diagramme de Gantt) . . . 6

7 Diagramme de déploiement **7**

8 Hypothèses **7**

8.1 Mobilisation des équipes . . . 7

8.2 Hypothèses de déroulement . . . 7

8.3 Risques . . . 8

1 Concept

1.1 Contexte

Les produits laitiers, les produits surgelés et de nombreux médicaments doivent rester dans une plage de température contrôlée, de l'entrepôt jusqu'au client. Beaucoup d'acteurs de la chaîne du froid utilisent encore des relevés manuels ou des enregistreurs consultés après coup. Une rupture de température peut donc n'être découverte qu'à la livraison, lorsque la marchandise est déjà perdue.

1.2 Problématique

Comment détecter rapidement une anomalie de température ou d'humidité, alerter immédiatement la bonne personne et garder une trace exploitable de l'incident ?

1.3 Ambitions

Développer une solution **Cloud of Things de supervision de la chaîne du froid** utilisant **Jakarta EE 11 & Node-RED / MQTT**. Des capteurs connectés mesurent la température, l'humidité et l'ouverture des portes ; une passerelle IoT transmet les mesures au cloud, qui les stocke, les analyse et déclenche les alertes.

La solution fournit :

— la **visualisation des données en temps réel** ;

— l'**historique**, des **analytiques** et des **indicateurs de performance** : temps passé hors plage, nombre d'incidents, délai moyen de réaction à une alerte ;

— des **alertes et notifications** pour chaque anomalie ;

— une **interface de surveillance et de configuration**.

Deux applications **PWA** sont développées :

— **front-office** : tableau de bord temps réel, historique, réception et acquittement des alertes ;

— **back-office** : gestion des capteurs, des sites, des seuils et des utilisateurs.

La prédiction n'est pas retenue en version 1 : les alertes reposent sur des règles mesurables. Elle pourra être étudiée si le temps et les données le permettent.

2 Clients, besoins et objectifs

2.1 Clients et utilisateurs

Client cible	**Usage principal**
Distributeurs alimentaires, laiteries	Suivi des chambres froides et des quais
Pharmacies, grossistes	Conservation des médicaments thermosensibles
Transporteurs frigorifiques	Surveillance des camions pendant la livraison

Trois profils utilisent la solution : le **responsable qualité** (alertes, analyse, rapports), le **chauffeur ou opérateur** (alertes de son véhicule ou de sa zone) et l'**administrateur** (configuration

dans le back-office).

2.2 Besoins fonctionnels

ID	Besoin	Priorité
EF-01	Mesurer en continu la température, l'humidité et l'ouverture de porte	Haute
EF-02	Afficher les mesures en temps réel	Haute
EF-03	Alerter en cas de dépassement de seuil : froid 0–4 °C, surgelé ≤ -18 °C, humidité 50–70 %	Haute
EF-04	Notifier le responsable qualité et le chauffeur concerné	Haute
EF-05	Acquitter et commenter une alerte	Moyenne
EF-06	Conserver l'historique des mesures et des incidents	Haute
EF-07	Afficher les KPI et exporter un rapport de conformité	Moyenne
EF-08	Gérer capteurs, sites, seuils et utilisateurs	Haute

2.3 Besoins non fonctionnels

— **Réactivité** : une alerte s'affiche moins de 10 secondes après la mesure ;

— **Sécurité** : MQTTS et HTTPS (TLS), accès par rôle, secrets hors des dépôts ;

— **Fiabilité** : la passerelle garde les mesures en mémoire en cas de coupure réseau ;

— **Ergonomie** : PWA installables sur mobile et simples à utiliser.

2.4 Occasions d'utilisation

— **Transport** : surveillance pendant la livraison, alerte au chauffeur ;

— **Stockage** : suivi permanent des chambres froides ;

— **Audit qualité** : consultation de l'historique et des incidents lors d'un contrôle.

2.5 Limites (hors périmètre)

La solution **surveille et informe** ; elle ne remplace ni l'équipement frigorifique ni les procédures de conformité. Sont exclus : le réglage du groupe froid, la gestion complète de flotte, la facturation, la certification réglementaire, les protocoles autres que MQTT (CoAP, AMQP) et la prédiction par IA.

3 Technologies choisies

Les technologies retenues couvrent la partie logicielle et la partie matérielle du projet.

3.1 Partie logicielle

Middleware

— **WildFly** : serveur d'applications open source compatible Jakarta EE 11, qui héberge l'API REST.

— **Mosquitto** : broker MQTT léger qui reçoit les mesures de la passerelle et les distribue au serveur ; configuré en MQTTS (TLS, port 8883).

— **Jakarta REST (JAX-RS)** : API Java pour exposer les services web REST consommés par les PWA.

Back-end

— **Jakarta EE 11** : CDI, Jakarta REST, Jakarta WebSocket (envoi des mesures et des alertes en temps réel) et Jakarta Security (authentification par rôle).

— **MongoDB** : base de données NoSQL orientée documents, adaptée aux séries de mesures, aux incidents et à la configuration.

— **MQTT** : protocole publish/subscribe léger utilisé entre la passerelle et le cloud.

Front-end

— **PWA (HTML5, CSS3, JavaScript)** : deux applications web installables sur mobile, avec mode hors ligne (Service Worker) et notifications push.

— **Chart.js** : graphiques temps réel et historiques du tableau de bord.

Intégration IoT

— **Node-RED** : outil de programmation par flux installé sur la passerelle ; il collecte les mesures des capteurs, les met en forme, les garde en mémoire en cas de coupure et les publie sur le broker.

— **Arduino IDE / C++** : programmation du firmware des ESP32.

Outils

— **GitHub** : versionnement du code et déploiement sur le cloud (`git clone` puis `git pull`).

— **Postman** : collection de tests de l'API REST.

— **Cloud IaaS (Ubuntu Linux)** : hébergement du broker, du serveur et de la base de données.

3.2 Partie matérielle

Composant	Rôle
Raspberry Pi 4	Passerelle IoT (Node-RED)
ESP32 (1 à 2)	Microcontrôleurs Wi-Fi reliés aux capteurs
DS18B20 (sonde étanche)	Température, y compris en surgelé (jusqu'à −55 °C)
DHT22	Température et humidité de l'air
Capteur magnétique de porte (reed)	Détection d'ouverture de porte
Buzzer et LED	Actionneurs : alerte locale en cas d'anomalie
Capteurs simulés	Script qui complète la flotte réelle pour les tests

4 Modèle commercial

4.1 Stratégie de commercialisation

— **Étude de marché** : les PME de la chaîne du froid (laiteries, distributeurs, pharmacies, transporteurs) sont peu équipées en supervision connectée ; les solutions existantes sont souvent consultées après coup ou trop coûteuses pour elles.

— **Différenciation** : traçabilité complète de chaque incident (horodatage, acquittement, commentaire), utile lors d'un audit, et abonnement sans investissement logiciel.

— **Jalons** : bêta-test avec un client pilote au T1 2027, lancement au T2 2027.

4.2 Politique marketing (4 P)

Politique	Choix
Produit	Kit capteurs ESP32, passerelle Raspberry Pi, plateforme cloud et deux PWA
Prix	Abonnement de 8 à 15 € par capteur et par mois selon le volume ; matériel vendu ou loué
Communication	Démonstration, vidéo, dossier de presse, LinkedIn et salons professionnels
Distribution	Vente directe aux PME et partenariats avec des frigoristes et logisticiens

Le prix, le client pilote et les dates de lancement sont des hypothèses de travail à valider.

5 Livrables

Livrable	Contenu
Cahier des charges	Le présent document
Cahier conceptuel	Besoins, cas d'utilisation, diagrammes UML, patrons de conception
Codes sources	Dépôts GitHub : API REST, PWA front-office et back-office, passerelle IoT
Déploiement	API REST sur Cloud IaaS, testable avec Postman ; PWA installées sur nos téléphones
Documentation	Documentation technique et manuel utilisateur
Communication	Supports marketing, dossier de presse, campagne publicitaire
Vidéos	Une vidéo de présentation de moins de 5 minutes par livrable

6 Contraintes

6.1 Temps, argent, objectifs

— **Temps** : livraison finale fin janvier 2027, en parallèle des autres modules ;

— **Argent** : matériel limité (1 Raspberry Pi, 1 à 2 ESP32), complété par des capteurs simulés ;

— **Objectifs** : priorité aux alertes fiables et au temps réel.

6.2 Méthodologie : eXtreme Programming

Itérations de deux semaines terminées par une démonstration, *user stories* issues des besoins EF-01 à EF-08, programmation en binôme sur les parties critiques, tests continus et intégration régulière sur GitHub. Le déploiement sur le cloud se fait uniquement depuis GitHub (`git clone` puis `git pull`).

6.3 Contraintes technologiques

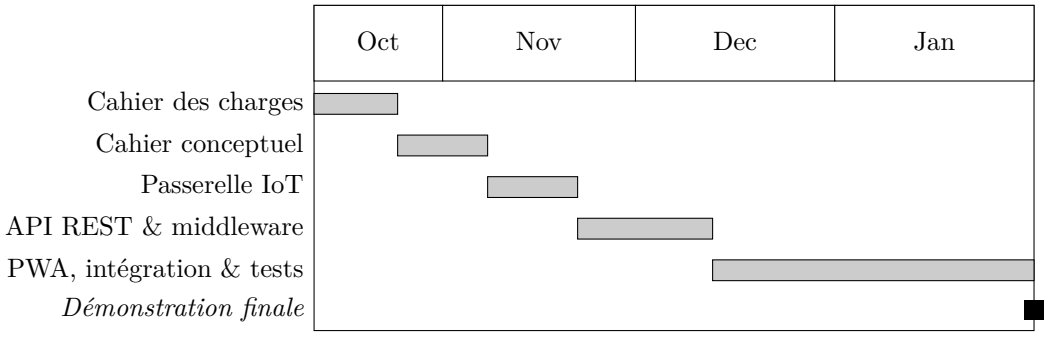
Le module impose Jakarta EE 11 pour le serveur, Node-RED et MQTT pour l'IoT, deux PWA pour les interfaces et un déploiement sur Cloud IaaS depuis GitHub. Le détail des technologies est donné dans la section 3.

6.4 Estimation des charges (CoCoMo+)

Les charges sont estimées par phase et par rôle, en **hommes-jours** (h-j).

Phase	**Période**	**Front**	**Middleware**	**Back / IoT**	**Total**
Cahier des charges	jusqu'au 24/10/2026	1	1	1	3
Cahier conceptuel	25/10 – 07/11/2026	3,5	4,5	4	12
Passerelle IoT & protocoles	08/11 – 21/11/2026	0	2	13	15
API REST & middleware	22/11 – 12/12/2026	0	15	5	20
PWA, intégration & tests	13/12/2026 – 31/01/2027	18	5	2	25
Total		**22,5**	**27,5**	**25**	**75**

6.5 Planning (diagramme de Gantt)



7 Diagramme de déploiement

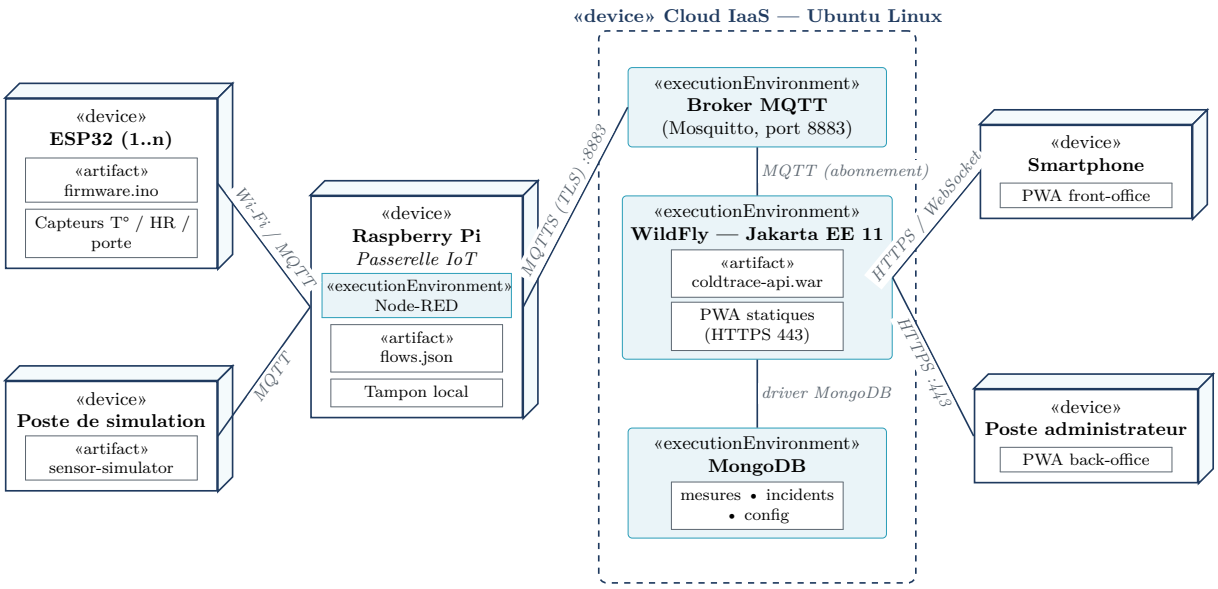


Figure 1 — Diagramme UML de déploiement de ColdTrace

Les mesures suivent le chemin capteur → passerelle → broker MQTT → API Jakarta EE → MongoDB, puis sont affichées dans les PWA. Seul MQTT (avec TLS) est utilisé en version 1 ; la passerelle conserve les mesures en cas de coupure réseau.

8 Hypothèses

8.1 Mobilisation des équipes

Équipe	Période	Travail
Back-end / IoT	08/11 – 21/11/2026	ESP32, simulateur, Node-RED, broker MQTT
Middleware	22/11 – 12/12/2026	API REST, alertes, MongoDB, déploiement cloud
Front-end	13/12/2026 – 31/01/2027	PWA front-office et back-office
Toute l'équipe	octobre et janvier	Cahiers, tests, documentation, vidéos

8.2 Hypothèses de déroulement

— des capteurs simulés complètent le matériel disponible ;

— une instance Cloud IaaS est accessible pendant toute la durée du projet ;

— une courte formation permet au client pilote de participer aux bêta-tests ;

— le planning suit les dates du module et les dépendances techniques : la passerelle précède l'API, qui précède les PWA.

8.3 Risques

Risque	Niveau	Réponse prévue
Matériel indisponible ou en panne	Élevé	Capteurs simulés prêts dès novembre
Intégration capteurs → cloud difficile	Élevé	Tester le flux complet le plus tôt possible
Prise en main de Jakarta EE 11	Moyen	Travail en binôme, prototype minimal d'abord
Accès au Cloud IaaS	Moyen	Provisionner l'instance dès le début
Charge des autres modules	Moyen	Marge dans le planning, priorisation des tâches